

FOREIGN TRADE UNIVERSITY
FALCUTY OF BANKING AND FINANCE

-----o0o-----

INDIVIDUAL REPORT
PIXIUFLOW WEBSITE
PROFIT TRACKING WEBSITE FOR MICRO-SMALL
BUSINESSES

Name : Dang Nhu Ngoc Thach

Student ID : 2312380032

Group : 02

Class : CTTTTC - K62

Course : NHA408E(2526-2)GD2.01

Lecturer : Phan Tran Trung Dzung, Assoc. Prof., Dr.

Hanoi, June 2026

TABLE OF CONTENTS

I. Who am I in this project?	2
1. My role in this project	2
2. Personal Highlight.....	2
II. What did I build?	4
1. Work actually done.....	4
2. Files and features contributed	6
III. How can you verify my work?	10
1. Evidence of contribution	10
2. How my work connects to the final product.....	11
IV. What did I learn?	12
1. Lessons Learned.....	12
2. Challenges and how I solved	13
V. What would I tell the future team?.....	14

TABLE OF FIGURES

Figure 1. Sales transaction entry	4
Figure 2. Deploy Pixiuflow on Railway.....	5
Figure 3. Personal Github Repository.....	10

I. Who am I in this project?

1. My role in this project

I served as the sole Fullstack Developer on PixiuFlow. I am responsible for translating the team's logic specifications, sitemap, and Figma mockups into a fully working Django web application.

My work covered the entire stack: database schema design, backend business logic, Django template rendering, custom CSS/JS, and deployment on Railway.

I did not start this project as a strong full-stack developer. I used AI coding tools (Claude, Codex, Gemini) as a learning and implementation assistant to generate initial code, explaining Django patterns, and helping debug errors. My role was to guide that process: describe requirements clearly, review the output, adjust it to fit the existing project structure, test the result, and make sure the final implementation matched the team's design and business logic.

2. Personal Highlight

The most distinctive part of my contribution was turning the team's pre-designed logic into a working system with the support of AI coding tools.

My teammates designed the business logic, the financial calculation rules, and the user flows. My role was to take those specifications and make them work in the actual application. This included understanding the expected behavior, asking AI for coding support, adjusting the code to fit the existing Django project structure, and testing whether the output worked correctly in the final product.

For example, sale entry in transaction entry page (/transactions/) is one of the clearest examples of how my work fit into the team's process.

What I received:

- UI/UX:
 - A Figma frame showing the layout: KPI cards at the top, a dashed order detail section with multi-row product entry, payment method dropdown, debt reminder date, and a total footer
- Logic flow:
 - Each sale line must capture product, unit price, quantity, and amount paid.

- A sale can only be recorded if the selected product has enough available inventory, so the user cannot sell more than the current stock.
- If the payment method is deferred payment, there should be receivables. In that case, the user must enter a debt reminder date, which will later support dashboard alerts when the receivable is close to its due date.
- Inventory must decrease by the quantity sold.
- The selling price should be auto-filled from the price previously entered by the user for that product, but the user should still be able to edit it manually when needed.

What I built from that:

- Starting from the Figma and logic specification, I used AI coding tools to support the implementation of the frontend screen with Django templates, CSS, and JavaScript. The main template was `sale_form.html`, which defined the page structure, form fields, tab layout, product rows, payment section, and total summary area.
- However, the AI-generated code did not always match the Figma design or the existing project structure correctly. Sometimes the layout was not aligned with the design, the spacing was different, the CSS did not match the intended visual style, or the generated code used field names and structures that were not suitable for my Django project. Because of that, I had to personally review the code, compare the output with the Figma screen, adjust the HTML structure, rewrite parts of the CSS, and test the page again until the interface looked closer to the intended design.
- The CSS handled the visual styling and layout, while JavaScript handled the interactive behavior such as adding/removing product rows and recalculating “Tổng doanh thu” in real time when the user changed unit price or quantity. For these JavaScript interactions, I also used AI support to generate the initial logic, but I still had to read the code, test the behavior, fix errors, and adjust it so it worked correctly with the actual form fields in my template.
- On the backend, I used AI support to help implement and debug the sale creation logic in `core/views.py`. The view processes the submitted form data, validates the selected product and quantity, checks whether the product has enough available inventory, creates the sale record, and applies the inventory update through the

existing sale/inventory logic. It also determines whether the sale is paid or deferred. If the customer chooses deferred payment, the receivable information and reminder date are stored so the dashboard can later show alerts when the debt is close to its due date.

Figure 1. Sales transaction entry page

II. What did I build?

1. Work actually done

On the frontend side, I built the main screens using Django templates, HTML, CSS, and vanilla JavaScript. I converted the Figma design into actual web pages, including the landing page, login/signup pages, dashboard, transaction forms, product and inventory page, transaction history, and report page. I also wrote the CSS structure for each major module, such as dashboard styling, transaction form styling, inventory layout, report/analytics layout, and responsive behavior.

I also implemented client-side interactions using JavaScript with AI support where needed. These included dynamic row adding/removing, real-time total calculation, tab switching, showing or hiding fields based on user choices, and passing chart data from Django views into the frontend for report visualization. After generating or adjusting the JavaScript, I tested whether the interactions worked correctly with the actual form fields and fixed errors when the behavior did not match the expected flow.

On the backend, I implemented the Django code needed to make the logic work in the system. This included writing and updating models, forms, views, URLs, and helper functions (core/models.py, core/forms.py, core/views.py, and core/urls.py). The business rules came from my logic teammate, while my contribution was to convert those rules into working Django code with the support of AI tools.

A key part of my work was connecting the UI with the backend. For example, the transaction entry page needed to accept product, quantity, selling price, and payment information from the user. My implementation made sure the form data could be submitted, validated, saved into the database, and reflected immediately in inventory, dashboard numbers, transaction history, and reports.

I also deployed the application to Railway so the project could be accessed and tested as a live web application, not only as local source code. For deployment, I connected the GitHub repository to Railway, configured the production environment variables, added the required production settings such as ALLOWED_HOSTS and CSRF_TRUSTED_ORIGINS, set up the Procfile, and checked the deployment logs to fix errors during the publishing process. After deployment, I tested the live website to make sure the main flows such as signup, login, dashboard viewing, transaction entry, inventory management, and reports could run properly in the production environment.

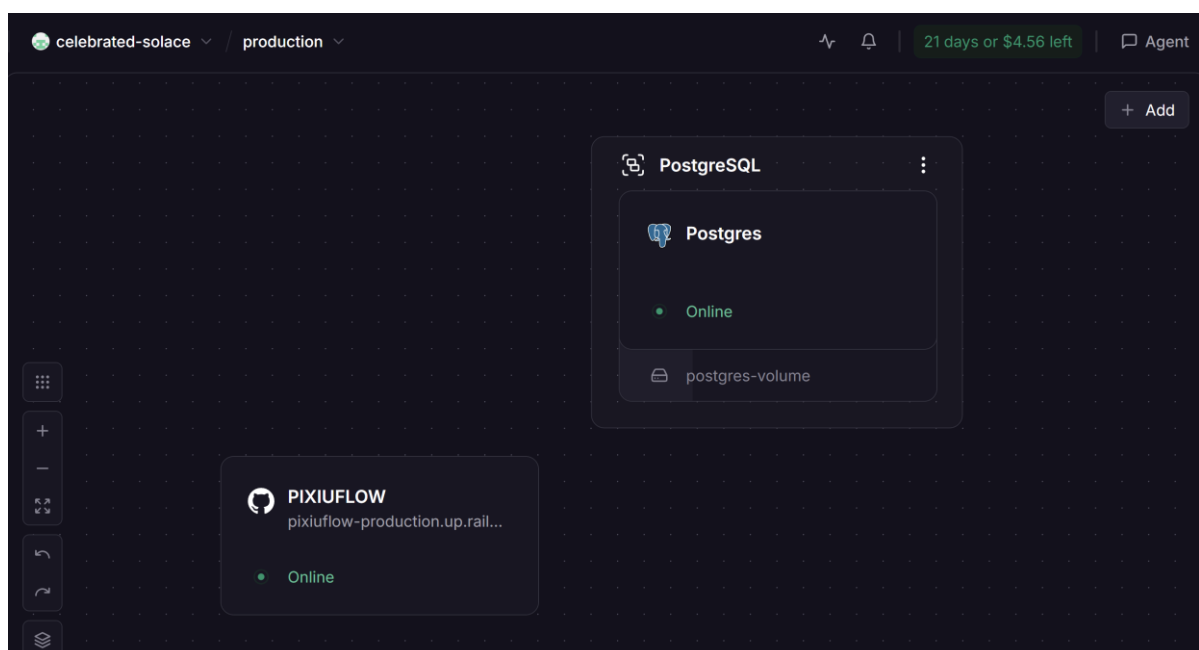


Figure 2. Deploy PixiuFlow on Railway

2. Files and features contributed

Main features

- User registration, login, logout, and password updates.
- Business owner profile management.
- Product, category, and inventory management.
- Sales, purchases, and expense recording.
- Bulk transaction entry.
- Payment status, debt, and due-date tracking.
- Revenue, profit, COGS, expense, and cash-flow calculations.
- Financial reports, charts, filters, and business insights.
- Low-stock and payment-due alerts.

Documentation and Demo

- Wrote the project overview, architecture, feature flows, and setup instructions in README.md
- Deployed the website on Railway and provided the live demo link to the team for testing, review, and presentation
- Live Demo: <https://pixiuflow-production.up.railway.app/>

Main files

Backend implementation files	
config/settings.py	Configures Django applications, database, templates, static files, authentication, and global context processors.
config/asgi.py / config /wsgi.py	Provide the server entry points for running and deploying the Django application.
core/context_processors.py	Gets the owner name from the logged-in user or session and sends it to all templates as owner_name.
core/models.py	Defines the database structure for products, categories, purchases, sales, expenses, and opening stock.

core/forms.py	Implemented Django forms for product input, sales entry, purchase entry, expense entry, opening stock setup, and transaction-related actions.
core/views.py	Contains the main business logic: authentication, inventory management, transactions, payment tracking, dashboard calculations, and financial reports.
core/urls.py	Defines the main project URLs, including login, logout, admin, and application routes.
core/utils.py	Provides reusable helpers for Vietnamese text processing and automatic product SKU generation.
core/templatetags/format_utils.py	Adds custom template filters for VND formatting, percentages, and text-encoding correction.
core/admin.py	Controls how application data can be managed through Django Admin.
core/apps.py	Defines the configuration for the Django core application.
core/migrations/	Stores database schema history, including payment tracking, user ownership, categories, stock dates, and product SKUs.
core/tests.py	Tests financial calculations, account settings, password changes, owner-name updates, and other important workflows.
Frontend template files	
core/templates/core/base.html	Provides the shared page layout, sidebar navigation, logout button, mobile menu, and common JavaScript.
core/templates/core/landing.html	Displays the public introduction page about PixiuFlow.
core/templates/core/signup.html	Provides account registration and initial business-profile setup.
core/templates/core/login.html	Provides the user login interface and error messages.

core/templates/core/account_settings.html	Allows users to update the business owner's name and change their password.
core/templates/core/dashboard.html	Displays revenue, expenses, profit, cash flow, alerts, and business performance summaries.
core/templates/core/sale_form.html	Handles sales, purchases, expenses, payment methods, and quick product creation.
core/templates/core/expense_form.html	Provides detailed forms for purchases and other business expenses.
core/templates/core/product_form.html	Allows users to create or edit products, categories, prices, and stock information.
core/templates/core/bulk_transaction_form.html	Built the bulk transaction input page so users can enter multiple records more efficiently.
core/templates/core/inventory.html	Displays products, stock levels, categories, inventory value, and low-stock warnings
core/templates/core/setup_products.html	Built the opening stock setup page for declaring products that already existed before using the app.
core/templates/core/transaction_history.html	Built the transaction history page with filters, search, status display, and action buttons.
core/templates/core/report.html	Displays financial analytics, charts, rankings, profit reports, expense analysis, and cash-flow reports.
CSS files	
styles.css	Imports the main shared CSS modules used across the application.

core/static/core/css /base.css	Defines global colors, typography, buttons, form controls, and common UI styles.
core/static/core/css /sidebar.css	Styles the desktop and mobile sidebar, navigation items, Settings button, and Logout button.
core/static/core/css /dashboard.css	Styles dashboard cards, statistics, alerts, charts, and the top bar..
core/static/core/css /transactions.css	Styles transaction forms, product selectors, dynamic rows, modals, and payment controls.
core/static/core/css /history.css	Styles transaction tables, filters, status labels, actions, and history summaries.
core/static/core/css /inventory.css	Styles inventory tables, category management, stock warnings, and inline editing.
core/static/core/css /analytics.css	Styles report tabs, charts, rankings, financial summaries, and insight panels.
core/static/core/css /responsive.css	Adapts layouts, forms, tables, navigation, and reports for tablets and mobile devices.
core/static/core/css /landing.css	Styles the public homepage, feature sections, banners, and calls to action.
core/static/core/css /onboarding.css	Styles the initial setup and product onboarding workflow.
core/static/core/css /signup.css	Styles the registration form, setup steps, and business profile interface.
core/static/core/css /auth.css and core/static/core/css /login.css	Style authentication layouts, login forms, and validation messages.

core/static/core/css /products.css	Styles product creation, category selection, pricing, and stock fields.
core/static/core/css /settings.css	Styles account information, owner-name updates, password forms, and success/error messages.

III. How can you verify my work?

1. Evidence of contribution

My contribution can be verified mainly through my personal GitHub repository, commit history, and the deployed Railway demo.

First, my personal GitHub repository contains the PixiuFlow source code that I developed during the project:

<https://github.com/thachnhungocdang/PIXIUFLOW>

This repository shows the full codebase of the application, including the Django backend, templates, CSS files, JavaScript interactions, and project configuration files. It can be used to review the files I worked on, such as core/models.py, core/forms.py, core/views.py, core/urls.py, the templates under core/templates/core/, and the CSS files under core/static/core/css/.

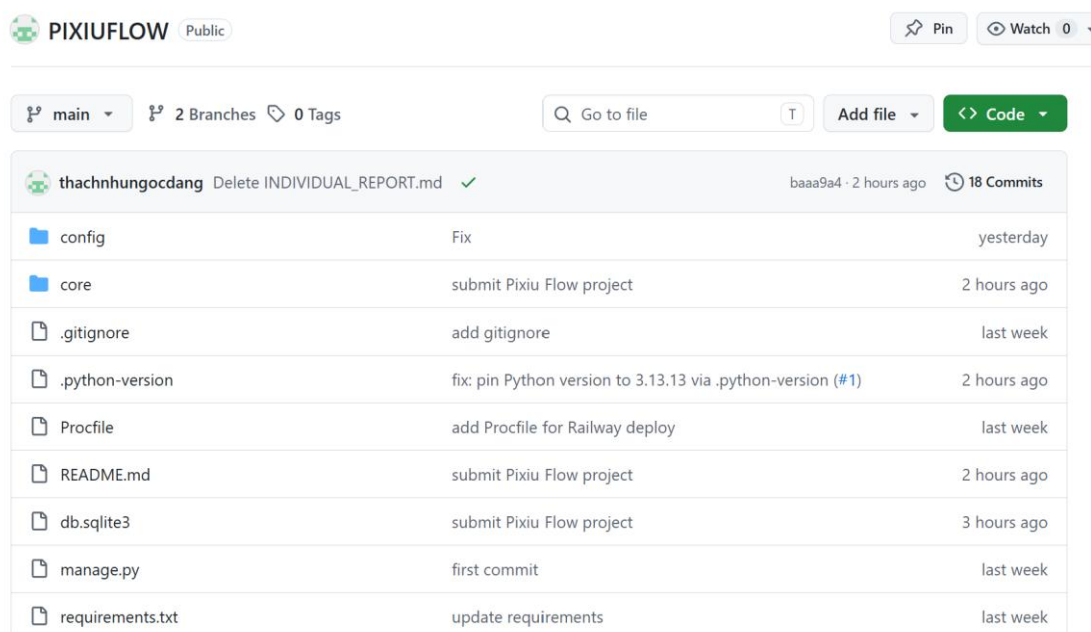


Figure 3. Personal Github Repository

Second, the commit history of my personal repository can be used to verify my implementation progress over time:

<https://github.com/thachnhungocdang/PIXIUFLOW/commits/main>

The commit history shows the changes made during development, including updates to production settings, deployment configuration, CSRF settings, onboarding/opening stock flow, dashboard/report improvements, and other implementation fixes. This provides evidence that the project was built and improved through actual code changes rather than only being uploaded at the end.

Third, the final working product can be checked through the Railway deployment:

<https://pixiuflow-production.up.railway.app/>

This demo link allows the product to be tested as a real web application. Through the deployed version, the main user flows can be verified, including registration/login, dashboard viewing, product and inventory management, transaction entry, transaction history, opening stock setup, and report analysis.

2. How my work connects to the final product

My work connects directly to the final PixiuFlow product because I implemented the code that turns the team's design and logic input into a working Django application.

The UI/UX teammate provided the Figma design, and the logic teammate defined the business rules. My role was to implement those inputs in code. I built the frontend screens using Django templates, HTML, vanilla CSS, and JavaScript, then connected them with backend processing in Django.

The screens I implemented form the main user journey of the product. Users can sign up, log in, set up initial products/opening stock, manage inventory, record transactions, view transaction history, and analyze business results through the dashboard and report pages.

On the backend, my implementation connects user actions to the database. When a user records a sale, purchase, expense, or opening stock item, the system processes the form, validates the input, saves the data, updates inventory, and reflects the result in the dashboard, history, and reports.

The deployment on Railway also shows that the project can be accessed and tested online. This supports the final demo and makes it easier for the people to verify the application without setting it up locally.

IV. What did I learn?

1. Lessons Learned

Through this project, I learned that building a working web application with the support of AI. At the beginning, I did not have strong full-stack development experience, especially with Django, backend logic, frontend templates, database handling, and deployment. Because of that, I used AI tools as a coding and learning assistant throughout the project. However, I did not use AI only to generate code and submit it directly. My role was to describe the requirements clearly, understand what the generated code was doing, test whether it worked, fix errors, adjust the implementation to match the team's design and logic, and connect different parts of the application into one working system. This process helped me learn full-stack development in a practical way.

One major lesson I learned is how to break a product requirement into smaller technical tasks. For example, a simple feature such as "record a sale" actually involves many parts: the HTML form, CSS layout, JavaScript interactions, Django view, form validation, database update, inventory update, payment status, dashboard numbers, and transaction history. Before this project, I did not fully understand how these parts connected. After implementing and debugging them, I learned how data flows from the user interface to the backend and then back to the screen.

Another lesson was how to use AI responsibly in development. AI helped me write code faster, explain errors, suggest Django patterns, and improve the UI. But AI also sometimes produced code that did not fully match my project structure or business requirement. Because of that, I had to review the output, compare it with my actual files, test it locally, check errors, and adjust the code. I learned that AI is useful, but it still requires human judgment, product understanding, and testing.

Finally, I learned how to deploy a Django project to Railway and connect it with GitHub.

This included setting production variables, fixing CSRF and allowed host issues, adding deployment configuration, and testing the live website.

Overall, this project helped me move from only understanding product ideas to understanding how a real web system is built, tested, debugged, and deployed. Even though I used AI support heavily, I learned by guiding the implementation, solving integration issues, and making sure the final product worked as expected.

2. Challenges and how I solved

The biggest challenge was that I did not start the project as a confident full-stack

developer. I was not fully familiar with how Django models, forms, views, templates, CSS, JavaScript, and deployment all work together. To solve this, I used AI tools to help me understand code structure, generate initial implementation ideas, explain errors, and suggest fixes. Then I tested the code, adjusted it to the real project, and checked whether each feature worked correctly.

One major challenge was that AI-generated code can sometimes be too broad or

inconsistent. To handle this, I learned to write more specific prompts. Instead of asking AI to “fix the page”, I described the exact file, expected behavior, validation rule, UI text, and business reason. This made the output more useful and reduced the risk of breaking existing features. In the end, the main challenge was not only writing code, but controlling and verifying the code quality while using AI assistance. I solved this by combining AI support with manual review, local testing, Git commit history, deployment testing, and continuous adjustment based on the final product requirements.

Another major technical challenge was debugging errors. Sometimes the code generated with AI did not run immediately because of missing variables, wrong field names, wrong routes, template errors, CSRF issues, or mismatches with the existing project structure. I solved these issues by reading the error messages, checking the related files, asking AI to explain the error, and then testing the fix again. This helped me become more familiar with Django debugging.

Another challenge was connecting the Figma design with the actual Django

implementation. The design showed what the screen should look like, but it did not explain how the data should move from the form to the backend. I solved this by breaking each screen into smaller parts: template structure, CSS layout, JavaScript behavior, form submission, view processing, and database update.

I **also faced deployment issues when publishing the project to Railway**. Some problems were related to environment variables, CSRF trusted origins, allowed hosts, migrations, and static files. I solved these step by step by checking Railway logs, updating the settings, adding the correct production variables, and testing the deployed site after each change.

V. What would I tell the future team?

If a future team builds a website related to finance, business management, or small-business operations, my main advice is to start by understanding the users before building features.

First, the team should spend enough time understanding how users actually think and work. For example, in our project, small business owners may not use accounting terms such as receivables, payables, COGS, or cash flow correctly. Therefore, the product should translate financial logic into simple language and clear actions. For example, instead of only showing numbers, the system should explain what the numbers mean and what the user should pay attention to.

Second, the product logic must be very clear from the beginning. In finance-related systems, similar actions can have very different meanings. For example, existing stock, new purchase, expense, unpaid sale, and cash received may all look related, but they affect reports differently. If the logic is unclear, the system may still work technically but produce misleading reports. Future teams should define these concepts carefully before coding.

Third, the team should design the user journey around real business decisions. A good financial website should not only let users enter data, but should help them answer important questions: Am I making a profit? Do I have enough cash? Which products are contributing the most? Which costs are increasing? What should I do next? These questions should guide the structure of dashboards, reports, alerts, and recommendations.

Finally, if the team uses AI tools to support development, they should still review and test the code carefully. AI can help generate code and explain errors, but the team must make sure the code matches the product logic, the target user's needs, and the final system structure.